ASSIGNMENT II

SUBMITTED BY

Keerthana

Roll No: 24CS097

ASSIGNMENT II

1.Scenario: The student admission portal requires a well-designed registration form with proper validation and styling.

Task:

1.Create HTML forms for student registration (name, roll number, email, course, file upload).
2.Apply CSS styles for layout, colors, and responsive design (e.g., styled input fields, buttons, error messages).
3.Implement client-side validation (JavaScript) for required fields, email format, and password strength.

HTML

```
<!DOCTYPE html>

<html lang="en">

<head>

   <meta charset="UTF-8">

   <meta name="viewport" content="width=device-width, initial-scale=1.0">

   <title>Student Registration Form</title>

   <link rel="stylesheet" href="style.css">

</head>

<body>

   <div class="container">

      <h2>Student Registration</h2>

      <p>Enter your details to register</p>

      <form id="registrationForm">

         <label for="name">Student Name</label>

         <input type="text" id="name" required>

         <label for="roll">Roll Number</label>

         <input type="text" id="roll" required>

         <label for="email">Email Address</label>
```

```
        <input type="email" id="email" required>
        <label for="course">Course</label>
        <select id="course" required>
          <option value="">Select Course</option>
          <option value="CSE">Computer Science</option>
          <option value="IT">Information Technology</option>
          <option value="ECE">Electronics</option>
          <option value="EEE">Electrical Engineering</option>
        </select>
        <label for="password">Password</label>
        <input type="password" id="password" required>
        <small id="error"></small>
        <label for="document">Upload File</label>
        <input type="file" id="document"
            accept=".jpg,.jpeg,.png,.pdf">
        <button type="submit">Register</button>
        <p id="message"></p>
      </form>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

CSS

```
body {
  font-family: Arial, sans-serif;
  background: #eaf0ff;
  margin: 0;
```

```
    padding: 30px 15px;
}
.container {
    max-width: 420px;
    margin: auto;
    padding: 25px;
    background: white;
    border-radius: 12px;
    box-shadow: 0 4px 15px #c5cde0;
}
h2 {
    text-align: center;
    color: #3730a3;
}
.container > p {
    text-align: center;
    color: #555;
}
label {
    display: block;
    margin-top: 15px;
    font-weight: bold;
}
input, select {
    width: 100%;
    padding: 10px;
    margin-top: 6px;
```

```
    border: 1px solid #bbb;
    border-radius: 5px;
    box-sizing: border-box;
}
input:focus, select:focus {
    outline: 2px solid #818cf8;
    border-color: #4f46e5;
}
button {
    width: 100%;
    padding: 12px;
    margin-top: 20px;
    background: #4f46e5;
    color: white;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    font-size: 16px;
}
button:hover {
    background: #3730a3;
}
#error {
    color: red;
    display: block;
    margin-top: 5px;
}
```

```
#message {
    color: green;
    font-weight: bold;
}
@media (max-width: 480px) {
    .container {
        padding: 18px;
    }
}
```

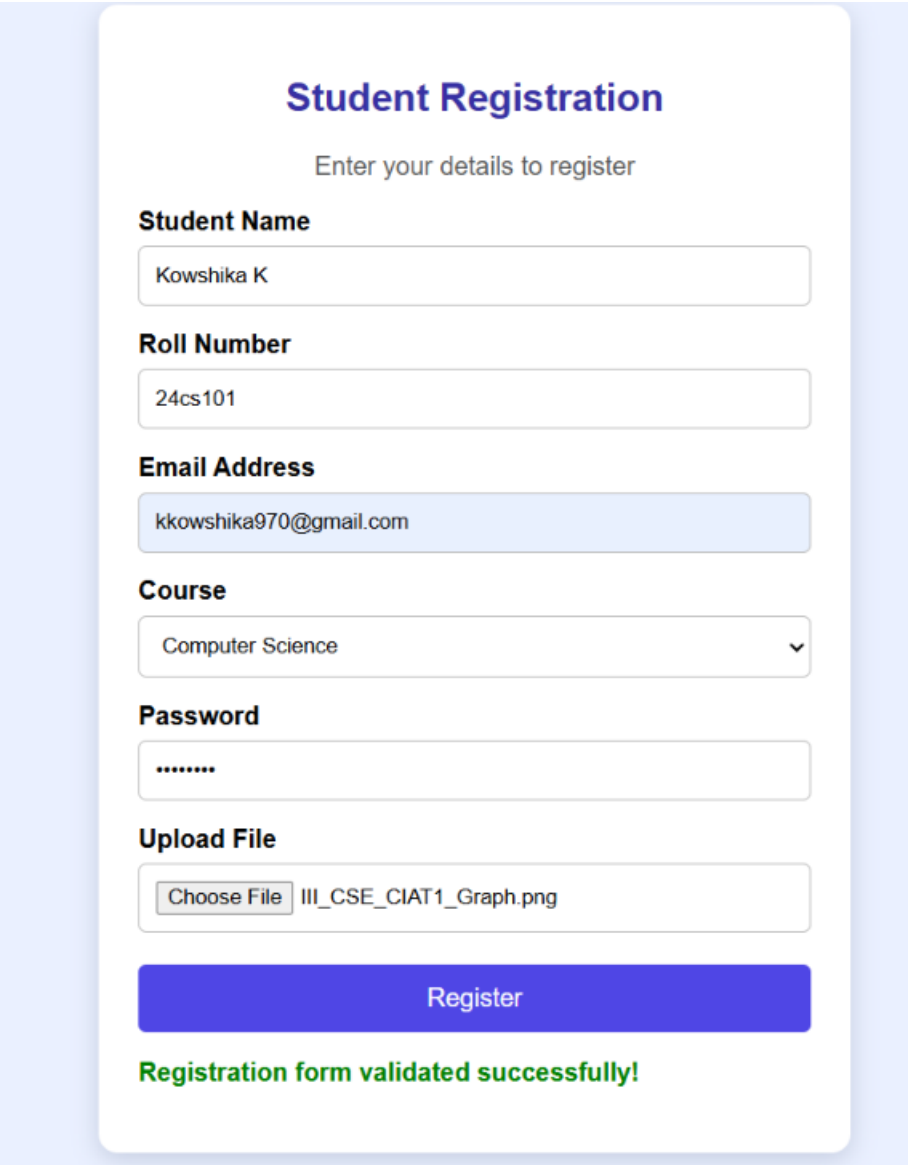
JAVA SCRIPT

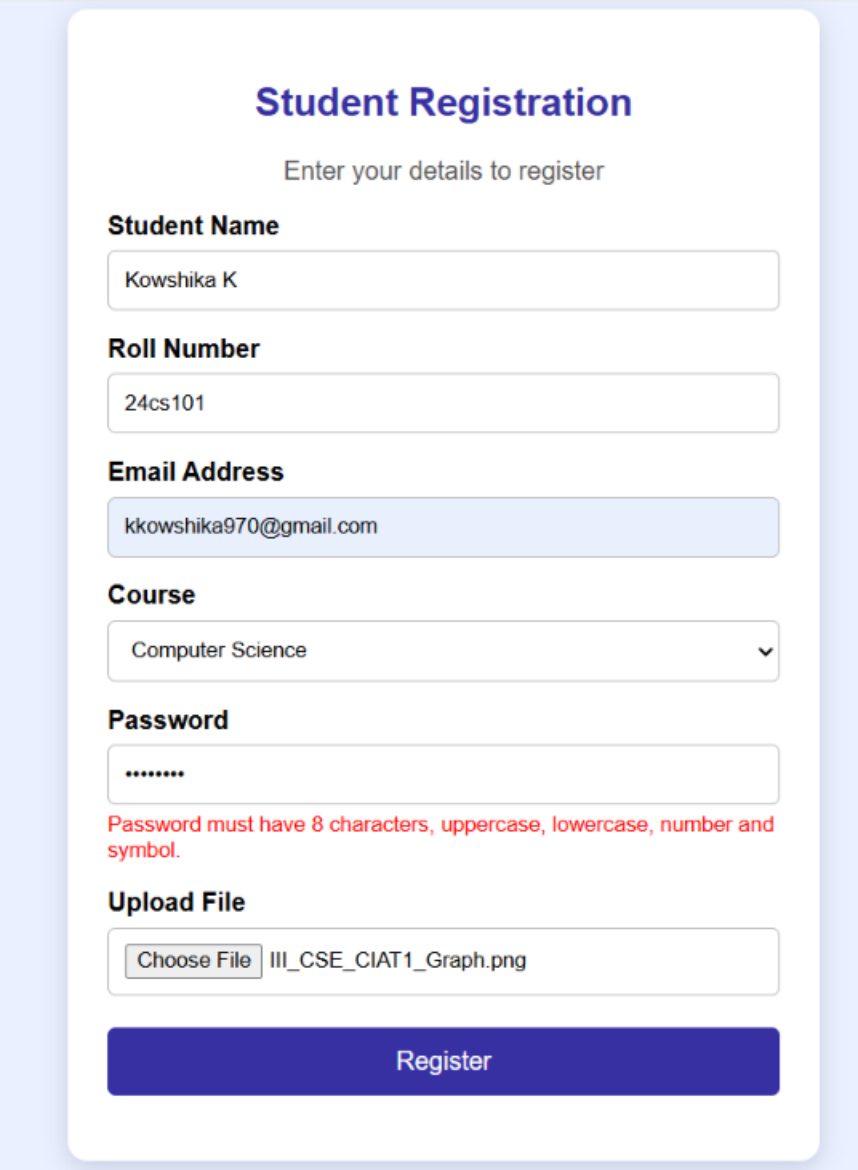
```
document.getElementById("registrationForm")
.addEventListener("submit", function(event) {
    // Prevent form submission until validation is complete
    event.preventDefault();
    let name = document.getElementById("name").value.trim();
    let roll = document.getElementById("roll").value.trim();
    let email = document.getElementById("email").value.trim();
    let course = document.getElementById("course").value;
    let password = document.getElementById("password").value;
    let file = document.getElementById("document").files[0];
    let error = document.getElementById("error");
    let message = document.getElementById("message");
    error.textContent = "";
    message.textContent = "";
    // Check required fields
    if (!name || !roll || !email || !course || !password) {
        error.textContent = "Please fill all required fields.";
```

```
    return;
}
// Validate email format
let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!emailPattern.test(email)) {
    error.textContent = "Please enter a valid email.";
    return;
}
// Validate password strength
let passwordPattern =
    /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^A-Za-z0-9]).{8,}$/;
if (!passwordPattern.test(password)) {
    error.textContent =
        "Password must have 8 characters, uppercase, lowercase, number and symbol.";
    return;
}
// Check uploaded file type and size
if (file) {
    let allowedTypes = [
        "image/jpeg",
        "image/png",
        "application/pdf"
    ];
    if (!allowedTypes.includes(file.type)) {
        error.textContent = "Only JPG, PNG and PDF files are allowed.";
        return;
    }
```

```
        if (file.size > 2 * 1024 * 1024) {
            error.textContent = "File size must not exceed 2 MB.";
            return;
        }
    }
    // Display success message
    message.textContent =
        "Registration form validated successfully!";
});
```

OUTPUT:





Explanation of Code with Inline Comments:

- <!DOCTYPE html> – Defines the document as HTML5.
- <html> – Represents the root element of the webpage.
- <head> – Contains webpage information and CSS links.
- <meta charset="UTF-8"> – Supports different characters.
- <meta name="viewport"> – Makes the webpage responsive on different devices.
- <title> – Sets the webpage title.

- <link rel="stylesheet"> – Connects the external CSS file.
- <body> – Contains the visible webpage content.
- – Creates the main form container.
- <h2> – Displays the heading “Student Registration”.
- <form> – Collects student information.
- <label> – Displays the name of each input field.
- <input type="text"> – Accepts the student's name and roll number.
- <input type="email"> – Accepts the student's email address.
- <select> and <option> – Allow the student to select a course.
- <input type="password"> – Accepts the student's password.
- <input type="file"> – Allows the student to upload a file.
- <button type="submit"> – Submits the form for validation.
- <small id="error"> – Displays validation error messages.
- <p id="message"> – Displays the success message.
- <script src="script.js"> – Connects the JavaScript file.

CSS Explanation

- body – Sets the font, background colour, and spacing.
- .container – Styles the form box with padding, rounded corners, and a shadow.
- h2 – Styles the form heading.
- label – Makes field labels clear and readable.
- input, select – Styles the input fields and dropdown list.
- :focus – Highlights a field when the user selects it.
- button – Styles the registration button.
- button:hover – Changes the button colour when the mouse moves over it.

- @media – Adjusts the layout for smaller screens.

JavaScript Explanation

- getElementById() – Selects HTML elements using their IDs.
- addEventListener() – Detects when the form is submitted.
- event.preventDefault() – Prevents the page from refreshing during validation.
- value.trim() – Retrieves input values and removes extra spaces.
- Regular expressions – Validate the email format and password strength.
- if conditions – Check required fields and file restrictions.
- file.type – Checks the uploaded file type.
- file.size – Checks the uploaded file size.
- textContent – Displays error and success messages.

CONCLUSION:

The Student Registration Form collects student details, validates the email and password, checks uploaded files, and displays appropriate messages. CSS makes the form attractive and responsive. This version performs client-side validation; it does not yet store student details in a database.

2.Scenario: The college admin panel must manage student records and their profile documents/images.

Task: Create PHP pages that:

1. **CRUD Operations (Add, Edit, Delete, View)** for student details (name, roll number, course, etc.) using MySQL.
2. **File Upload Handling** for profile images and documents (e.g., ID proof, certificates).
3. **Store Metadata in MySQL** — file path, file type, upload date linked to the student record.
4. **Display Uploaded Files** when viewing student details.
5. **Validation & Error Handling** — check file size, type, and prevent overwriting.

PROGRAM CODE:

Start XAMPP

Open XAMPP Control Panel and click Start for:

```
Apache
MySQL
```

Both services should show Running

Test XAMPP

Open the browser and visit:

```
http://localhost
```

Then test phpMyAdmin:

```
http://localhost/phpmyadmin
```

Create the Project Folder

Create the following folder:

```
C:\xampp\htdocs\admin
```

Create the Project Files

```
q2_php_admin/
├── db.php
├── index.php
├── add.php
```

```
├── edit.php
├── delete.php
├── view.php
├── style.css
└── uploads/
    ├── images/
    └── documents/
```

Create the Database

Open:

```
http://localhost/phpmyadmin
```

Click New and create a database named:

```
admin
```

Create the Users Table

```
CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    roll_number VARCHAR(30) NOT NULL UNIQUE,
    course VARCHAR(100) NOT NULL,
    email VARCHAR(100),
    phone VARCHAR(15),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE student_files (
    id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT NOT NULL,
    original_name VARCHAR(255) NOT NULL,
    stored_name VARCHAR(255) NOT NULL,
    file_path VARCHAR(500) NOT NULL,
    file_type VARCHAR(100) NOT NULL,
    upload_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (student_id) REFERENCES students(id)
        ON DELETE CASCADE
);
```

dp.php

<?php

```
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

$conn = new mysqli("localhost", "root", "", "admin");
$conn->set_charset("utf8mb4");
?>
```

index.php

```
<?php
require "db.php";

$result = $conn->query(
    "SELECT * FROM students ORDER BY id DESC"
);
?>
<!DOCTYPE html>
<html>
<head>
    <title>Student Admin Panel</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
    <h1>Student Admin Panel</h1>
    <a class="btn" href="add.php">+ Add Student</a>
    <table>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Roll Number</th>
            <th>Course</th>
            <th>Actions</th>
        </tr>
        <?php while ($row = $result->fetch_assoc()) { ?>
        <tr>
```

```
            <td><?= (int)$row['id'] ?></td>
            <td><?= htmlspecialchars($row['name']) ?></td>
            <td><?= htmlspecialchars($row['roll_number']) ?></td>
            <td><?= htmlspecialchars($row['course']) ?></td>
            <td>
                <a href="view.php?id=<?= (int)$row['id'] ?>">View</a> |
                <a href="edit.php?id=<?= (int)$row['id'] ?>">Edit</a> |
                <form method="POST" action="delete.php"
                      class="inline-form"
                      onsubmit="return confirm('Delete this student and their files?');">
                    <input type="hidden" name="id"
                           value="<?= (int)$row['id'] ?>">
                    <button type="submit" class="delete-btn">Delete</button>
                </form>
            </td>
        </tr>
        <?php } ?>
    </table>
    <?php if ($result->num_rows === 0) { ?>
        <p>No student records found. Click Add Student to begin.</p>
    <?php } ?>
</div>
</body>
</html>
```

add.php

```
<?php
require "db.php";
$message = "";
$maxImageSize = 2 * 1024 * 1024;
$maxDocSize = 5 * 1024 * 1024;
function saveUpload($field, $folder, $allowed, $maxSize) {
    if (!isset($_FILES[$field]) ||
        $_FILES[$field]['error'] === UPLOAD_ERR_NO_FILE) {
```

```
        return null;
    }
    $file = $_FILES[$field];
    if ($file['error'] !== UPLOAD_ERR_OK) {
        throw new Exception("Upload failed. Please try again.");
    }
    if ($file['size'] > $maxSize) {
        throw new Exception("File exceeds the allowed size.");
    }
    $finfo = new finfo(FILEINFO_MIME_TYPE);
    $mime = $finfo->file($file['tmp_name']);
    if (!isset($allowed[$mime])) {
        throw new Exception("Invalid file type.");
    }
    $extension = $allowed[$mime];
    $storedName = bin2hex(random_bytes(16)) . "." . $extension;
    $relativePath = "uploads/" . $folder . "/" . $storedName;
    $fullPath = __DIR__ . "/" . $relativePath;
    if (!move_uploaded_file($file['tmp_name'], $fullPath)) {
        throw new Exception("Could not save uploaded file.");
    }
    return [
        "original" => basename($file['name']),
        "stored" => $storedName,
        "path" => $relativePath,
        "mime" => $mime,
        "full" => $fullPath
    ];
}
$imageTypes = [
    "image/jpeg" => "jpg",
    "image/png" => "png",
    "image/webp" => "webp"
```

```
];
$documentTypes = [
    "application/pdf" => "pdf",
    "application/msword" => "doc",
    "application/vnd.openxmlformats-officedocument.wordprocessingml.document"
        => "docx"
];
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $savedFiles = [];
    $transactionStarted = false;
    try {
        $name = trim($_POST["name"] ?? "");
        $roll = trim($_POST["roll_number"] ?? "");
        $course = trim($_POST["course"] ?? "");
        $email = trim($_POST["email"] ?? "");
        $phone = trim($_POST["phone"] ?? "");
        if ($name === "" || $roll === "" || $course === "") {
            throw new Exception("Name, roll number and course are required.");
        }
        if (strlen($name) > 100 || strlen($roll) > 30 ||
            strlen($course) > 100 || strlen($email) > 100 ||
            strlen($phone) > 15) {
            throw new Exception("One or more fields are too long.");
        }
        if ($email !== "" && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
            throw new Exception("Enter a valid email address.");
        }
        // Validate both files before inserting the student.
        $image = saveUpload(
            "profile_image", "images", $imageTypes, $maxImageSize
        );
        if ($image) {
            $savedFiles[] = $image["full"];
```

```
}
$document = saveUpload(
    "document", "documents", $documentTypes, $maxDocSize
);
if ($document) {
    $savedFiles[] = $document["full"];
}
$conn->begin_transaction();
$transactionStarted = true;
$stmt = $conn->prepare(
    "INSERT INTO students
     (name, roll_number, course, email, phone)
     VALUES (?, ?, ?, ?, ?)"
);
$stmt->bind_param("sssss", $name, $roll, $course, $email, $phone);
$stmt->execute();
$studentId = $conn->insert_id;
foreach ([$image, $document] as $file) {
    if ($file === null) {
        continue;
    }
    $stmt = $conn->prepare(
        "INSERT INTO student_files
         (student_id, original_name, stored_name, file_path, file_type)
         VALUES (?, ?, ?, ?, ?)"
    );
    $stmt->bind_param(
        "issss",
        $studentId,
        $file["original"],
        $file["stored"],
        $file["path"],
        $file["mime"]
```

```
            );
            $stmt->execute();
        }
        $conn->commit();
        $transactionStarted = false;
        header("Location: view.php?id=" . $studentId);
        exit;
    } catch (Throwable $e) {
        if ($transactionStarted) {
            $conn->rollback();
        }
        foreach ($savedFiles as $path) {
            if (is_file($path)) {
                unlink($path);
            }
        }
        if ($e instanceof mysqli_sql_exception &&
            (int)$e->getCode() === 1062) {
            $message = "This roll number already exists.";
        } else {
            $message = $e instanceof mysqli_sql_exception
                ? "Database operation failed. Please check your database setup."
                : $e->getMessage();
        }
    }
}
?>
<!DOCTYPE html>
<html>
<head>
    <title>Add Student</title>
    <link rel="stylesheet" href="style.css">
</head>
```

```
<body>
<div class="container">
  <h1>Add Student</h1>
  <?php if ($message !== "") { ?>
    <p class="error"><?= htmlspecialchars($message) ?></p>
  <?php } ?>
  <form method="POST" enctype="multipart/form-data">
    <label>Student Name *</label>
    <input type="text" name="name" maxlength="100" required>
    <label>Roll Number *</label>
    <input type="text" name="roll_number" maxlength="30" required>
    <label>Course *</label>
    <input type="text" name="course" maxlength="100" required>
    <label>Email</label>
    <input type="email" name="email" maxlength="100">
    <label>Phone Number</label>
    <input type="text" name="phone" maxlength="15">
    <label>Profile Image (JPG, PNG, WebP; max 2 MB)</label>
    <input type="file" name="profile_image"
        accept=".jpg,.jpeg,.png,.webp">
    <label>ID Proof / Certificate (PDF, DOC, DOCX; max 5 MB)</label>
    <input type="file" name="document"
        accept=".pdf,.doc,.docx">
    <button type="submit">Save Student</button>
    <a class="btn secondary" href="index.php">Back</a>
  </form>
</div>
</body>
</html>
```

edit.php

```
<?php
require "db.php";
```

```
$id = filter_input(INPUT_GET, "id", FILTER_VALIDATE_INT);
if (!$id || $id < 1) {
    exit("Invalid student ID.");
}
$stmt = $conn->prepare("SELECT * FROM students WHERE id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();
$student = $stmt->get_result()->fetch_assoc();
if (!$student) {
    exit("Student not found.");
}
$message = "";
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $name = trim($_POST["name"] ?? "");
    $roll = trim($_POST["roll_number"] ?? "");
    $course = trim($_POST["course"] ?? "");
    $email = trim($_POST["email"] ?? "");
    $phone = trim($_POST["phone"] ?? "");
    if ($name === "" || $roll === "" || $course === "") {
        $message = "Name, roll number and course are required.";
    } elseif (
        strlen($name) > 100 || strlen($roll) > 30 ||
        strlen($course) > 100 || strlen($email) > 100 ||
        strlen($phone) > 15
    ) {
        $message = "One or more fields are too long.";
    } elseif ($email !== "" &&
            !filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $message = "Enter a valid email address.";
    } else {
        try {
            $stmt = $conn->prepare(
                "UPDATE students
```

```
                SET name = ?, roll_number = ?, course = ?,
                    email = ?, phone = ?
                WHERE id = ?"
            );
            $stmt->bind_param(
                "ssssi",
                $name, $roll, $course, $email, $phone, $id
            );
            $stmt->execute();
            header("Location: view.php?id=" . $id);
            exit;
        } catch (mysqli_sql_exception $e) {
            $message = ((int)$e->getCode() === 1062)
                ? "That roll number already exists."
                : "Unable to update the student.";
        }
    }
}
?>
<!DOCTYPE html>
<html>
<head>
    <title>Edit Student</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
    <h1>Edit Student</h1>
    <?php if ($message !== "") { ?>
        <p class="error"><?= htmlspecialchars($message) ?></p>
    <?php } ?>
    <form method="POST">
        <label>Student Name *</label>
```

```
        <input type="text" name="name" maxlength="100"
            value="<?= htmlspecialchars($student['name']) ?>" required>
        <label>Roll Number *</label>
        <input type="text" name="roll_number" maxlength="30"
            value="<?= htmlspecialchars($student['roll_number']) ?>" required>
        <label>Course *</label>
        <input type="text" name="course" maxlength="100"
            value="<?= htmlspecialchars($student['course']) ?>" required>
        <label>Email</label>
        <input type="email" name="email" maxlength="100"
            value="<?= htmlspecialchars($student['email'] ?? '') ?>">
        <label>Phone Number</label>
        <input type="text" name="phone" maxlength="15"
            value="<?= htmlspecialchars($student['phone'] ?? '') ?>">
        <button type="submit">Update Student</button>
        <a class="btn secondary" href="view.php?id=<?= $id ?>">Cancel</a>
    </form>
</div>
</body>
</html>
```

delete.php

```
<?php
require "db.php";
if ($_SERVER["REQUEST_METHOD"] !== "POST") {
    header("Location: index.php");
    exit;
}
$id = filter_input(INPUT_POST, "id", FILTER_VALIDATE_INT);
if (!$id || $id < 1) {
    exit("Invalid student ID.");
}
try {
    $conn->begin_transaction();
```

```
    $stmt = $conn->prepare(
        "SELECT file_path FROM student_files WHERE student_id = ?"
    );
    $stmt->bind_param("i", $id);
    $stmt->execute();
    $result = $stmt->get_result();
    $paths = [];
    while ($row = $result->fetch_assoc()) {
        $path = $row["file_path"];
        // Only permit files within the two designated upload folders.
        if (
            preg_match(
                '#\Auploads/(?:images|documents)/[a-f0-
9]{32}\.(?:jpg|png|webp|pdf|doc|docx)\z#i',
                $path
            )
        ) {
            $paths[] = __DIR__ . DIRECTORY_SEPARATOR .
                str_replace("/", DIRECTORY_SEPARATOR, $path);
        }
    }
    $stmt = $conn->prepare(
        "DELETE FROM student_files WHERE student_id = ?"
    );
    $stmt->bind_param("i", $id);
    $stmt->execute();

    $stmt = $conn->prepare(
        "DELETE FROM students WHERE id = ?"
    );
    $stmt->bind_param("i", $id);
    $stmt->execute();
    $conn->commit();
    // Remove physical files after the database transaction succeeds.
```

```
    foreach ($paths as $path) {
        if (is_file($path)) {
            unlink($path);
        }
    }
    header("Location: index.php");
    exit;
} catch (Throwable $e) {
    try {
        $conn->rollback();
    } catch (Throwable $ignored) {
        // Transaction may already have ended.
    }
    http_response_code(500);
    exit("Unable to delete the student. Please check the database.");
}
?>
```

view.php

```
<?php
require "db.php";
$id = filter_input(INPUT_GET, "id", FILTER_VALIDATE_INT);
if (!$id || $id < 1) {
    exit("Invalid student ID.");
}
$stmt = $conn->prepare("SELECT * FROM students WHERE id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();
$student = $stmt->get_result()->fetch_assoc();
if (!$student) {
    exit("Student not found.");
}
$stmt = $conn->prepare(
    "SELECT * FROM student_files WHERE student_id = ? ORDER BY id DESC"
```

```
);
$stmt->bind_param("i", $id);
$stmt->execute();
$files = $stmt->get_result();
function h($value) {
    return htmlspecialchars((string)$value, ENT_QUOTES, "UTF-8");
}
?>
<!DOCTYPE html>
<html>
<head>
    <title>View Student</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
    <h1>Student Details</h1>
    <p><strong>Name:</strong> <?= h($student["name"]) ?></p>
    <p><strong>Roll Number:</strong> <?= h($student["roll_number"]) ?></p>
    <p><strong>Course:</strong> <?= h($student["course"]) ?></p>
    <p><strong>Email:</strong> <?= h($student["email"] ?: "Not provided") ?></p>
    <p><strong>Phone:</strong> <?= h($student["phone"] ?: "Not provided") ?></p>
    <hr>
    <h2>Uploaded Files</h2>
    <?php
    $hasFiles = false;
    while ($file = $files->fetch_assoc()) {
        $hasFiles = true;
        $path = $file["file_path"];
        $type = $file["file_type"];
        // Display only paths inside the expected upload folders.
        if (!preg_match(
            '#\Auploads/(?:images|documents)/[a-f0-9]{32}\.(?:jpg|png|webp|pdf|doc|docx)\z#i',
```

```
        $path
    )) {
        continue;
    }
    $isImage = in_array(
        $type,
        ["image/jpeg", "image/png", "image/webp"],
        true
    );
?>
    <div class="file-card">
        <p>
            <strong>File:</strong>
            <?= h($file["original_name"]) ?>
        </p>
        <p><strong>Type:</strong> <?= h($type) ?></p>
        <p>
            <strong>Uploaded:</strong>
            <?= h($file["upload_date"]) ?>
        </p>
        <?php if ($isImage) { ?>
            <img
                src="<?= h($path) ?>"
                alt="Student profile image"
                class="profile-image"
            >
        <?php } ?>
        <p>
            <a href="<?= h($path) ?>"
               target="_blank"
               rel="noopener noreferrer">
                Open / View File
            </a>
```

```
            </p>
        </div>
    <?php } ?>
    <?php if (!$hasFiles) { ?>
        <p>No files have been uploaded for this student.</p>
    <?php } ?>
    <hr>
    <a class="btn" href="edit.php?id=<?= (int)$student["id"] ?>">
        Edit Student
    </a>
    <a class="btn secondary" href="index.php">
        Back to Student List
    </a>
</div>
</body>
</html>
```

Style.css

```
body {
    font-family: Arial, sans-serif;
    background: #f2f5fa;
    margin: 0;
    padding: 20px;
    color: #333;
}
.container {
    max-width: 1000px;
    width: 90%;
    margin: 30px auto;
    padding: 25px;
    background: white;
    border-radius: 10px;
    box-shadow: 0 3px 12px #ccc;
}
```

```
h1 {
    text-align: center;
    color: #243b64;
}
h2 {
    color: #1769aa;
}
label {
    display: block;
    margin-top: 15px;
    margin-bottom: 5px;
    font-weight: bold;
}
input {
    display: block;
    width: 100%;
    padding: 10px;
    border: 1px solid #ccc;
    border-radius: 5px;
    box-sizing: border-box;
}
button, .btn {
    display: inline-block;
    background: #1769aa;
    color: white;
    padding: 10px 16px;
    margin-top: 15px;
    border: none;
    border-radius: 5px;
    text-decoration: none;
    cursor: pointer;
    font-size: 14px;
}
```

```
.secondary {
    background: #666;
}
.delete-btn {
    background: #dc3545;
    padding: 5px 9px;
    margin: 0;
}
.inline-form {
    display: inline;
}
table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 20px;
}
th, td {
    padding: 12px;
    border: 1px solid #ddd;
    text-align: center;
}
th {
    background: #1769aa;
    color: white;
}
td a {
    color: #1769aa;
}
.error {
    background: #ffe6e6;
    color: #a00000;
    padding: 10px;
    border-radius: 5px;
```

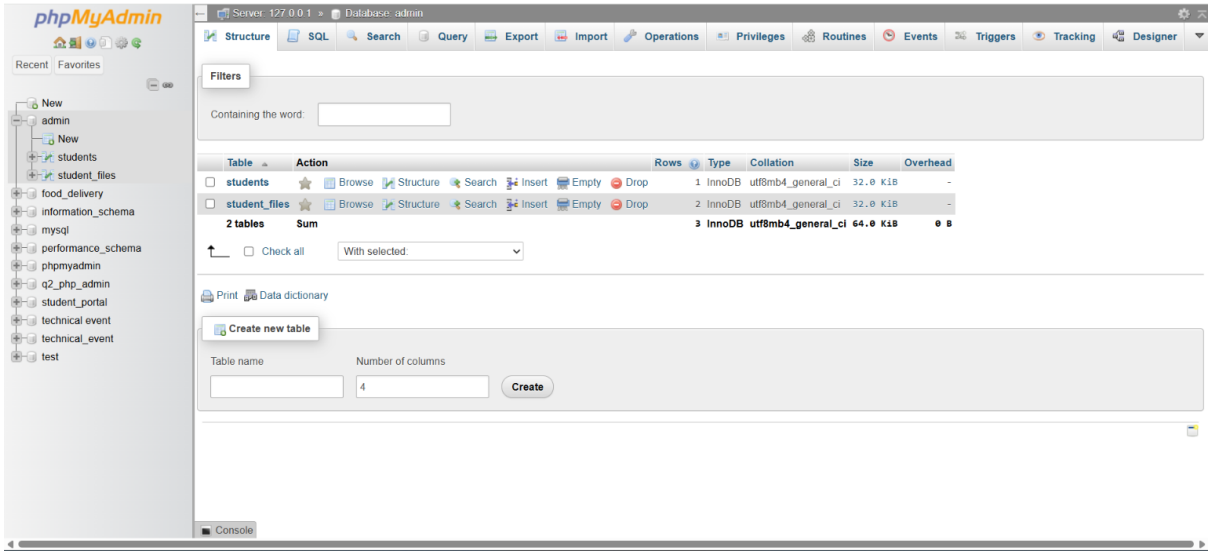
```
}
.file-card {
    padding: 15px;
    margin: 15px 0;
    border: 1px solid #ddd;
    border-radius: 8px;
}
.profile-image {
    display: block;
    width: 160px;
    max-height: 180px;
    object-fit: contain;
    margin: 10px 0;
    border: 1px solid #ddd;
    border-radius: 5px;
}
@media (max-width: 650px) {
    .container {
        width: auto;
        overflow-x: auto;
    }
    table {
        min-width: 650px;
    }
}
```

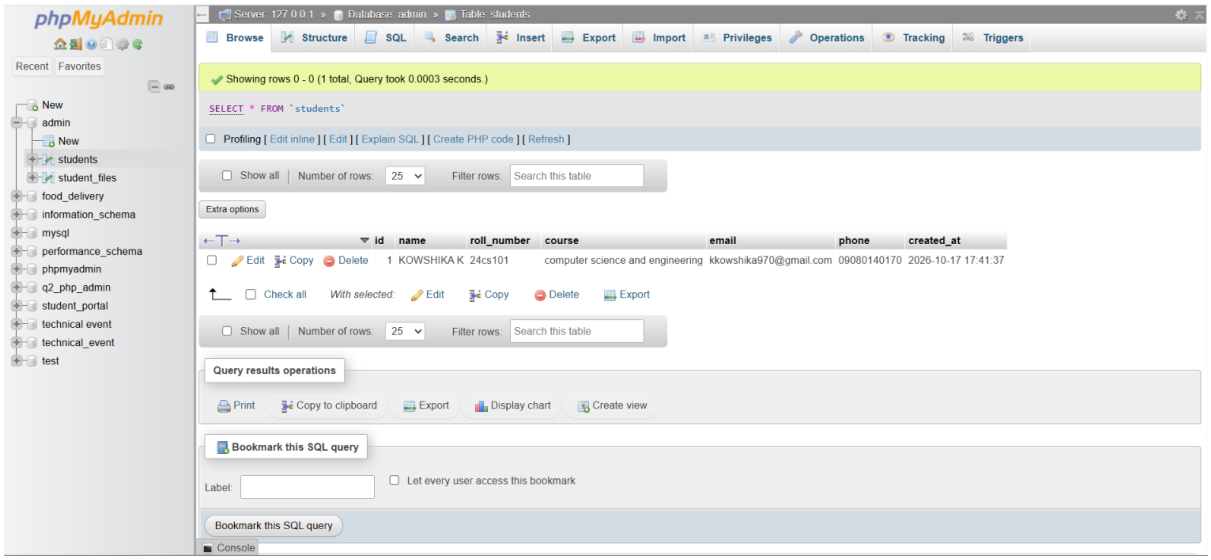
Execute the Project

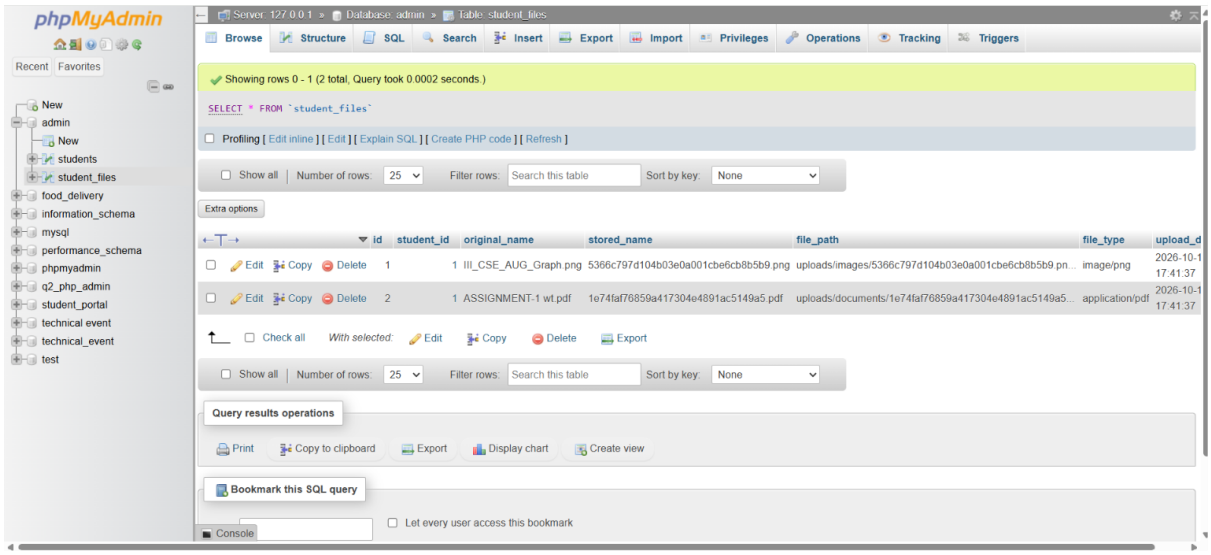
Make sure Apache and MySQL are running, then open:

```
http://localhost/admin/
```

OUTPUT:


phpMyAdmin
Server: 127.0.0.1 » Database: admin
Structure SQL Search Query Export Import Operations Privileges Routines Events Triggers Tracking Designer
Filters
Containing the word:
Table Action Rows Type Collation Size Overhead
students Browse Structure Search Insert Empty Drop 1 InnoDB utf8mb4_general_ci 32.0 KiB -
student_files Browse Structure Search Insert Empty Drop 2 InnoDB utf8mb4_general_ci 32.0 KiB -
2 tables Sum 3 InnoDB utf8mb4_general_ci 64.0 KiB 0 B
Check all With selected:
Print Data dictionary
Create new table
Table name Number of columns
4
Create
Console


phpMyAdmin
Server: 127.0.0.1 » Database: admin » Table: students
Browse Structure SQL Search Insert Export Import Privileges Operations Tracking Triggers
Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)
SELECT * FROM `students`
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all Number of rows: 25 Filter rows: Search this table
Extra options
id name roll_number course email phone created_at
Edit Copy Delete 1 KOWSHIKA K 24cs101 computer science and engineering kkowshika970@gmail.com 09080140170 2026-10-17 17:41:37
Check all With selected: Edit Copy Delete Export
Query results operations
Print Copy to clipboard Export Display chart Create view
Bookmark this SQL query
Label:
Let every user access this bookmark
Bookmark this SQL query
Console


phpMyAdmin
Server: 127.0.0.1 » Database: admin » Table: student_files
Browse Structure SQL Search Insert Export Import Privileges Operations Tracking Triggers
Showing rows 0 - 1 (2 total, Query took 0.0002 seconds.)
SELECT * FROM `student_files`
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all Number of rows: 25 Filter rows: Search this table Sort by key: None
Extra options
id student_id original_name stored_name file_path file_type upload_d
Edit Copy Delete 1 1 III_CSE_AUG_Graph.png 5366c797d104b03e0a001cbe6cb8b5b9.png uploads/images/5366c797d104b03e0a001cbe6cb8b5b9.pn... image/png 2026-10-1 17:41:37
Edit Copy Delete 2 1 ASSIGNMENT-1 wt.pdf 1e74faf76859a417304e4891ac5149a5.pdf uploads/documents/1e74faf76859a417304e4891ac5149a5... application/pdf 2026-10-1 17:41:37
Check all With selected: Edit Copy Delete Export
Query results operations
Print Copy to clipboard Export Display chart Create view
Bookmark this SQL query
Let every user access this bookmark
Console

Student Admin Panel

+ Add Student

ID	Name	Roll Number	Course	Actions

No student records found. Click Add Student to begin.

Add Student

Student Name *

KOWSHIKA K

Roll Number *

24cs101

Course *

computer science and engineering

Email

kkowshika970@gmail.com

Phone Number

9080140170

Profile Image (JPG, PNG, WebP; max 2 MB)

Choose File III_CSE_AUG_Graph.png

ID Proof / Certificate (PDF, DOC, DOCX; max 5 MB)

Choose File ASSIGNMENT-1 wt.pdf

Save Student Back

Student Details

Name: KOWSHIKA K

Roll Number: 24cs101

Course: computer science and engineering

Email: kkowshika970@gmail.com

Phone: 9080140170

Uploaded Files

File: ASSIGNMENT-1 wt.pdf

Type: application/pdf

Uploaded: 2026-10-17 18:25:21

Open / View File

File: III_CSE_AUG_Graph.png

Type: image/png

Uploaded: 2026-10-17 18:25:21



Open / View File

Edit Student | Back to Student List

Student Admin Panel

+ Add Student

ID	Name	Roll Number	Course	Actions
2	KOWSHIKA K	24cs101	computer science and engineering	View | Edit | Delete

Explanation of Code with Inline Comments:

PHP Code Explanation

- <?php ... ?> – Defines the beginning and end of a PHP code block.
- require "db.php" – Connects the PHP page to the MySQL database.
- new mysqli() – Establishes a connection with the MySQL database.
- $conn->query() – Executes an SQL query to retrieve student records.
- $_SERVER["REQUEST_METHOD"] – Checks whether the form was submitted using the POST method.
- $_POST – Retrieves the student details submitted through the form.
- $_FILES – Contains information about uploaded images and documents.
- trim() – Removes unnecessary spaces from the beginning and end of input values.
- filter_var() – Validates the email address format.
- mysqli_report() – Enables detailed MySQL error reporting.
- prepare() – Creates a prepared SQL statement for safer database operations.
- bind_param() – Binds user input values to the SQL statement.
- execute() – Executes the prepared SQL statement.
- INSERT INTO – Adds new student details to the database.
- SELECT – Retrieves student information from the database.
- UPDATE – Modifies existing student details.
- DELETE – Removes student records from the database.
- fetch_assoc() – Retrieves a database row as an associative array.
- htmlspecialchars() – Prevents user-provided text from being interpreted as HTML.
- header("Location: ...") – Redirects the browser to another PHP page.
- exit() – Stops further PHP execution.
- begin_transaction() – Starts a database transaction.
- commit() – Saves the database transaction.
- rollback() – Reverses database changes when an operation fails.
- catch (Throwable $e) – Handles errors and exceptions.
- password_hash() – Not used in this project because student records do not include passwords.

File Upload Handling Explanation

- enctype="multipart/form-data" – Enables file uploads through an HTML form.
- $_FILES – Provides uploaded file details such as name, size, type, and temporary path.

- UPLOAD_ERR_OK – Indicates that the file upload completed successfully.
- UPLOAD_ERR_NO_FILE – Indicates that no file was selected.
- filesize / $_FILES["..."]["size"] – Checks the uploaded file size.
- finfo(FILEINFO_MIME_TYPE) – Detects the file type from its contents.
- $imageTypes – Specifies the permitted profile image formats.
- $documentTypes – Specifies the permitted document formats.
- random_bytes() – Generates random data used to create unique filenames.
- bin2hex() – Converts random bytes into hexadecimal characters.
- move_uploaded_file() – Moves an uploaded file into the destination folder.
- basename() – Extracts the filename without its directory path.
- unlink() – Deletes a physical file from the server.
- $file["path"] – Stores the relative path of the uploaded file.
- $file["mime"] – Stores the detected MIME type in MySQL.
- student_id – Links uploaded file metadata to the corresponding student record.

MySQL Database Explanation

- CREATE TABLE – Creates a database table.
- INT AUTO_INCREMENT PRIMARY KEY – Generates a unique numeric ID for each record.
- VARCHAR() – Stores variable-length text.
- NOT NULL – Requires a value in a column.
- UNIQUE – Prevents duplicate roll numbers.
- TIMESTAMP DEFAULT CURRENT_TIMESTAMP – Automatically records the date and time.
- FOREIGN KEY – Establishes a relationship between the student and uploaded file tables.
- ON DELETE CASCADE – Automatically deletes related file metadata when a student record is deleted.
- file_path – Stores the location of the uploaded file.
- file_type – Stores the detected file type.
- upload_date – Stores the file upload date and time.

HTML Code Explanation

- <!DOCTYPE html> – Defines the document as HTML5.
- <html> – Represents the root element of the webpage.
- <head> – Contains webpage information and CSS links.

- <title> – Sets the webpage title.
- <link rel="stylesheet"> – Connects the external CSS file.
- <body> – Contains the visible webpage content.
- – Creates the main content container.
- <h1> and <h2> – Display headings.
- <form> – Collects student information.
- <label> – Displays the name of each input field.
- <input type="text"> – Accepts the student's name, roll number, course, and phone number.
- <input type="email"> – Accepts the student's email address.
- <input type="file"> – Allows the administrator to upload a profile image or document.
- required – Ensures that a required field is filled in before submitting the form.
- maxlength – Limits the number of characters entered.
- <button type="submit"> – Submits the form for processing.
- <table>, <tr>, <th>, and <td> – Display student records in tabular format.
- <a href="..."> – Navigates to the View, Edit, or Add Student page.
- <img> – Displays the uploaded profile image.
- htmlspecialchars() – Safely displays student details and filenames.

CSS Explanation

- body – Sets the font, background colour, and spacing.
- .container – Styles the content box with padding, rounded corners, and a shadow.
- h1 – Styles the main heading.
- label – Makes form labels clear and readable.
- input – Styles the input fields.
- button – Styles the form buttons.
- .btn – Styles navigation links as buttons.
- .secondary – Gives secondary buttons a different background colour.
- table – Sets the table width and combines table borders.
- th, td – Adds padding, borders, and text alignment to table cells.
- .error – Displays error messages with a coloured background.
- .file-card – Creates a separate section for each uploaded file.
- .profile-image – Controls the size and appearance of profile images.
- @media – Adjusts the layout for smaller screens.

CRUD Operations Explanation

- Create – Adds new student records using add.php and an SQL INSERT statement.
- Read – Displays student records using index.php and view.php with SQL SELECT statements.
- Update – Modifies student information using edit.php and an SQL UPDATE statement.
- Delete – Removes student information and uploaded files using delete.php and SQL DELETE statements.

Validation and Error Handling Explanation

- Required field checks – Ensure the name, roll number, and course are entered.
- Email validation – Checks whether the email address has a valid format.
- File size validation – Rejects images larger than 2 MB and documents larger than 5 MB.
- File type validation – Allows only the specified image and document formats.
- Unique filenames – Prevent existing uploaded files from being overwritten.
- Duplicate roll number handling – Displays an error if the roll number already exists.
- Database error handling – Catches database exceptions and displays an appropriate error message.
- Transaction handling – Uses commit() and rollback() to maintain database consistency.

CONCLUSION:

Thus, the Student Admin Panel was successfully developed using PHP and MySQL. It supports adding, viewing, editing, and deleting student records. It also allows uploading profile images and documents while storing their metadata in the database. File validation and error handling help ensure reliable data management.

3.Scenario: The front-end portal must allow users to view student profiles along with their uploaded images/documents.

Task: Develop Servlet pages that:

1. **Connect to MySQL using JDBC** to fetch student details and associated file paths.
2. **Retrieve and stream images/documents** from the folder where they are stored.
3. **Display student profiles dynamically** in an HTML/JSP page with <img> tags pointing to the Servlet.
4. **Implement looping and conditional logic** to list multiple students and their files.
5. **Ensure session handling** so that only logged-in users can access the portal securely.

1. PHP, Servlet WEB-INF Files and Other Files (.html, .css, .js)

Files used for (Java Servlet/JSP Portal):

- java-portal/src/main/java/com/studentportal/config/DBConnection.java – JDBC factory, dynamic port fallback
- java-portal/src/main/java/com/studentportal/dao/AdminDAO.java– BCrypt admin authentication DAO
- java-portal/src/main/java/com/studentportal/dao/StudentDAO.java – Student list and profile retrieval
- java-portal/src/main/java/com/studentportal/dao/DocumentDAO.java – Document metadata DAO
- java-portal/src/main/java/com/studentportal/servlet/LoginServlet.java – /login controller
- java-portal/src/main/java/com/studentportal/servlet/StudentListServlet.java – /students controller
- java-portal/src/main/java/com/studentportal/servlet/StudentProfileServlet.java – /profile controller
- java-portal/src/main/java/com/studentportal/servlet/DocumentStreamServlet.java – Binary file streamer
- java-portal/src/main/java/com/studentportal/filter/AuthFilter.java – Session authorization filter

- java-portal/src/main/webapp/WEB-INF/web.xml – Servlet and filter mappings
- java-portal/src/main/webapp/WEB-INF/jsp/login.jsp – Login page JSP
- java-portal/src/main/webapp/WEB-INF/jsp/students.jsp – JSTL student list table
- java-portal/src/main/webapp/WEB-INF/jsp/profile.jsp – Student profile and document links

LoginServlet.java

```
package com.studentportal.servlet;
import com.studentportal.dao.AdminDAO;
import com.studentportal.model.AdminUser;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
import java.io.IOException;
import java.sql.SQLException;
/** Handles administrator authentication into the Java Portal. */
@WebServlet(name = "LoginServlet", urlPatterns = {"/login"})
public class LoginServlet extends HttpServlet {
private AdminDAO adminDAO;
@Override public void init() throws ServletException {
this.adminDAO = new AdminDAO(); // Initialise AdminDAO on servlet startup
}
@Override
protected void doGet(HttpServletRequest req, HttpServletResponse resp)
throws ServletException, IOException {
// If already logged in, redirect to student list
HttpSession session = req.getSession(false);
if (session != null && session.getAttribute("adminUser") != null) {
resp.sendRedirect(req.getContextPath() + "/students");
return;
}
// Show logout banner if redirected after logout
```

```
if ("1".equals(req.getParameter("logout")))
req.setAttribute("successMessage", "You have been safely logged out.");
req.getRequestDispatcher("/WEB-INF/jsp/login.jsp").forward(req, resp);
}
@Override
protected void doPost(HttpServletRequest req, HttpServletResponse resp)
throws ServletException, IOException {
req.setCharacterEncoding("UTF-8");
String username = req.getParameter("username");
String password = req.getParameter("password");
if (username == null || username.trim().isEmpty() ||
password == null || password.isEmpty()) {
req.setAttribute("errorMessage", "Please provide both username and password.");
req.getRequestDispatcher("/WEB-INF/jsp/login.jsp").forward(req, resp);
return;
}
try {
AdminUser admin = adminDAO.authenticate(username, password);
if (admin != null) {
// Invalidate old session to prevent session fixation attacks
HttpSession old = req.getSession(false);
if (old != null) old.invalidate();
// Create a new session and store the admin object
HttpSession newSession = req.getSession(true);
newSession.setAttribute("adminUser", admin);
newSession.setMaxInactiveInterval(1800); // 30-minute timeout
resp.sendRedirect(req.getContextPath() + "/students");
} else {
req.setAttribute("errorMessage",
"Invalid administrator username or password.");
req.getRequestDispatcher("/WEB-INF/jsp/login.jsp").forward(req, resp);
}
} catch (SQLException e) {
```

```
req.setAttribute("errorMessage",
"Database error: Could not verify credentials.");
req.getRequestDispatcher("/WEB-INF/jsp/login.jsp").forward(req, resp);
        }
    }
}
```

StudentDAO.java

```
/**
*Data Access Object for Student and Document retrieval.
* Uses PreparedStatement and try-with-resources for safe, leak-free queries.
*/
/**
 * Data Access Object for Student and Document retrieval.
 * Uses PreparedStatement and try-with-resources for safe, leak-free queries.
 */
public List<Student> getAllStudents(String search) throws SQLException {
   String sql = "SELECT s.*, " +
                "(SELECT COUNT(*) FROM student_documents d " +
                "WHERE d.student_id = s.id) AS doc_count " +
                "FROM students s " +
                "ORDER BY s.registration_date DESC";

   try (Connection conn = DBConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {
       List<Student> list = new ArrayList<>();
       while (rs.next()) {
          list.add(mapStudent(rs)); // Map each row to a Student POJO
       }
       return list;
   }
}
```

2. DB Schema Script

It uses the same shared student_portal database as CO3 and CO4. The Java application connects via JDBC to the students, admin_users, and student_documents tables. No additional tables are required

-- CO5 connects to the same shared student_portal database.

-- The Java DBConnection.java factory connects with JDBC using:

-- jdbc:mysql://localhost:3307/student_portal?useSSL=false&serverTimezone=UTC

-- Port fallback: tries 3307 first (XAMPP MariaDB), then 3306 (default MySQL).

-- All three tables (admin_users, students, student_documents) are reused.

-- No additional schema changes are required for the Java portal.

-- DBConnection.java JDBC URL pattern:

-- "jdbc:mysql://127.0.0.1:" + port + "/student_portal?useSSL=false&serverTimezone=UTC"
–

-- PreparedStatement example from StudentDAO.java:

-- String sql = "SELECT * FROM students ORDER BY registration_date DESC";

-- PreparedStatement ps = conn.prepareStatement(sql);

-- ResultSet rs = ps.executeQuery()

3. Screenshots of Output in Browser

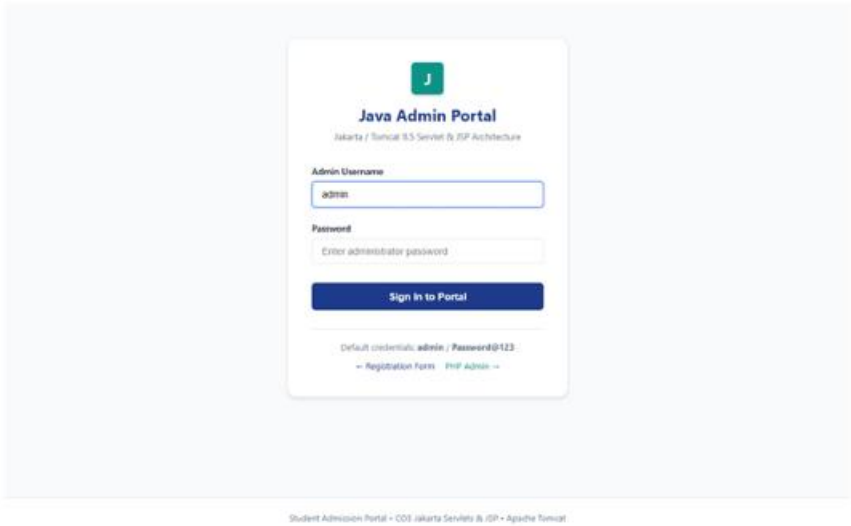


Figure 12: Java Admin Portal Login Page (Tomcat port 8082)

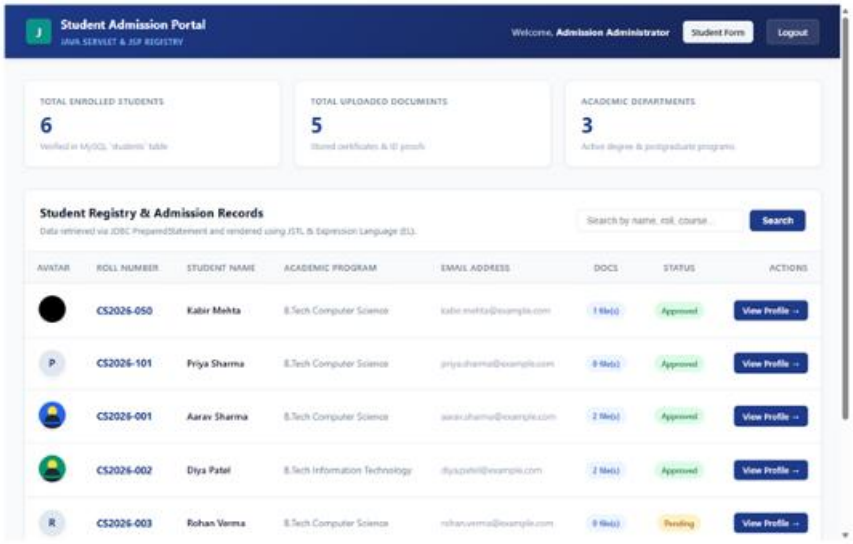


Figure 13: JSTL Student Registry Table with Dynamic KPIs

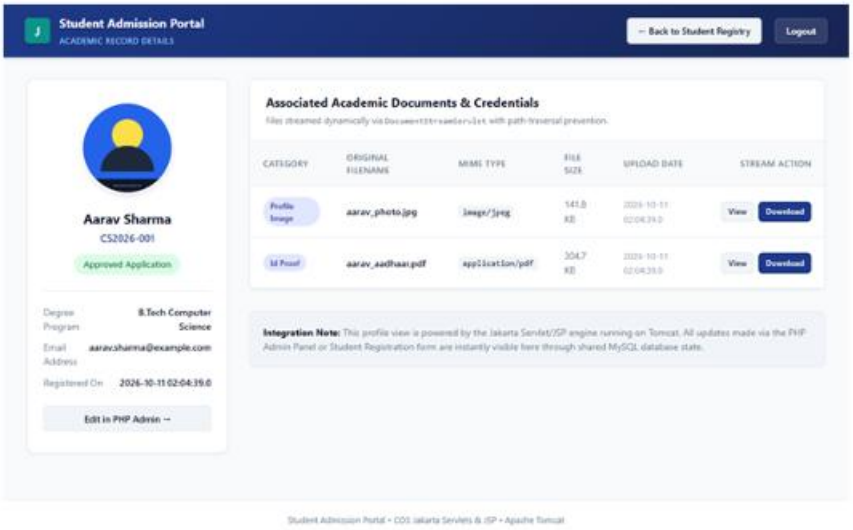


Figure 14: Student Profile View with Streamed Avatar and Documents



Figure 15: Binary Image Served via DocumentStreamServlet

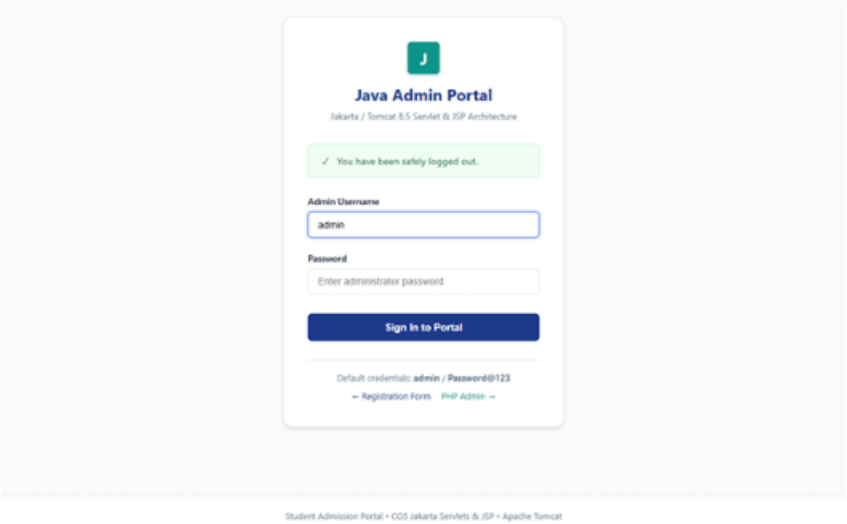


Figure 16: Session Invalidation and Logout Verification Banner

5. Explanation of Code with Inline Comments

DBConnection.java:

A static JDBC connection factory that attempts to connect to MariaDB on port 3307 first. If the connection fails (e.g., XAMPP MariaDB is on port 3307 while standard MySQL is on 3306), it automatically retries on port 3306. The factory is called with try-with-resources in every DAO to guarantee that connections are closed after each query.

AdminDAO.java:

Fetches the admin row from admin_users by username using a PreparedStatement. The stored BCrypt hash is then verified using the BCrypt.checkpw() method (jBCrypt library), which safely compares the submitted plain-text password against the $2y$ hash without revealing timing information

LoginServlet.java:

Mapped to the URL pattern /login via the @WebServlet annotation. doGet() renders the JSP login page; doPost() reads the credentials, calls adminDAO.authenticate(), and on success invalidates the old session (session fixation prevention) and creates a fresh HttpSession storing the AdminUser object with a 30-minute inactive timeout.

StudentListServlet.java:

Mapped to /students. Retrieves the full student list from StudentDAO.getAllStudents() and forwards it as a request attribute named "students" to students.jsp. The JSTL tag iterates over the list to render the table rows.

DocumentStreamServlet.java:

Mapped to /docs. Accepts a stored_filename query parameter and resolves the absolute path under the uploads/ directory using Paths.get().normalize(). It verifies the resolved path starts with the canonical uploads root (path traversal defense). Files are then read and written to the response OutputStream in 8 KB chunks, with Content-Type and Content-Length headers set correctly so the browser can display or download the file.

AuthFilter.java:

Implements javax.servlet.Filter and is mapped to /students, /profile, and /docs URL patterns. For every incoming request it checks whether the HttpSession contains a valid adminUser attribute. If not present, it redirects the unauthenticated request to the /login page, preventing any access to protected pages without a valid session.

5. Brief Conclusion

It successfully demonstrates an enterprise-grade Java web application deployed on Apache Tomcat 8.5+ using Servlets 3.1, JSP, JSTL, and JDBC connected to the shared MySQL database. The LoginServlet, StudentListServlet, StudentProfileServlet, and DocumentStreamServlet collectively provide secure admin authentication, dynamic JSTL-rendered student registry views, and binary document streaming with path-traversal protection. The AuthFilter ensures that every protected resource is inaccessible without an active authenticated session, meeting all CO5 enterprise security requirements.

Repository Link:

https://github.com/Kowshi1731/ASSIGNMENT---2